

Project 2 – Part 2: DNS Client from Scratch

Authors: Georges Rizk (919388525), Shayan Khosh Keifi (923713063)

1. Output Screenshots

```
Querying 198.41.0.4 for wikipedia.org
-----
NS : a2.org.afiliast-nst.info
NS : b2.org.afiliast-nst.org
NS : d0.org.afiliast-nst.org
NS : a0.org.afiliast-nst.info
NS : b0.org.afiliast-nst.org
NS : c0.org.afiliast-nst.info
A : 199.249.112.1
AAAA : 2001:500:40::1
A : 199.249.120.1
AAAA : 2001:500:48::1
A : 199.19.57.1
AAAA : 2001:500:f::1
A : 199.19.56.1
AAAA : 2001:500:e::1
A : 199.19.54.1
AAAA : 2001:500:c::1
A : 199.19.53.1
AAAA : 2001:500:b::1
RTT: 32 ms

Querying 199.249.112.1 for wikipedia.org
-----
NS : ns0.wikimedia.org
NS : ns1.wikimedia.org
NS : ns2.wikimedia.org
A : 208.80.154.238
AAAA : 2620:0:861:53::1
A : 208.80.153.231
AAAA : 2620:0:860:53::1
A : 198.35.27.27
AAAA : 2a02:ec80:53::1
RTT: 59 ms

Querying 208.80.154.238 for wikipedia.org
-----
A : 198.35.26.96
RTT: 86 ms

Making HTTP request to 198.35.26.96
-----
HTTP/1.1 301 Moved Permanently
RTT: 62 ms
```

2. Creating the DNS Packet: How was it structured?

The DNS request packet is constructed manually according to the DNS protocol specification. A DNS query packet consists of a header and a question section.

- Header (12 bytes):
- Transaction ID used to match requests and responses
- Flags indicating the query type
- Question count
- Answer count
- Authority count
- Additional record count

In our implementation the transaction ID is randomly generated and the recursion desired flag is set to 0 so that the program performs iterative DNS resolution instead of relying on recursive resolvers.

The question section contains the encoded domain name (QNAME), the query type (QTYPE), and the query class (QCLASS). The domain name is encoded using DNS wire format where each label is preceded by its length and the name is terminated with a zero byte.

3. Parsing each response: How was the information extracted?

After sending a DNS query, the program receives a response packet containing several sections including the header, question, answer, authority, and additional records. The response is parsed to extract the relevant DNS records.

The program decodes domain names using a function that supports DNS compression pointers. These pointers allow the response to reuse previously stored domain name strings within the packet.

Each resource record is parsed to extract fields such as name, type, class, time-to-live (TTL), and record data. The parsed records are stored in dictionaries that allow the program to identify A records, NS records, CNAME records, and other record types.

The iterative resolver follows the referral chain by starting with root servers, then querying TLD servers, and finally querying authoritative name servers until the final A record is obtained.

4. HTTP request: How was the HTTP request created? What was received?

After resolving the domain name to an IP address, the program creates a TCP connection to the resolved IP address on port 80. A raw HTTP GET request is constructed manually using the following format:

- GET / HTTP/1.1
- Host: <domain>
- Connection: close

The program sends this request to the server and receives the HTTP response. The first line of the response is extracted and printed, which typically contains the HTTP status code such as 'HTTP/1.1 200 OK'.

5. External Resources

The following resources were used during the development of this project:

- Python socket documentation: This documentation was used to understand how to implement TCP and UDP in python.
- Python struct module documentation.
- RFC 1035 – Domain Names Implementation and Specification.
- ChatGPT was used for conceptual explanations of networking concepts and debugging guidance.
- Lecture 10 – IP Protocol & Fragmentation
- Lecture 11 – Forwarding, ICMP, Naming
- Lecture 12 – ICMP, Naming, NAT
- Lecture 13 – Project 2 Concepts
- Lecture 14 – NAT and Routing
- Lecture 15 – Intra-Domain Routing
- The lecture on naming also explains how the Domain Name System maps hostnames to IP addresses, which is the concept implemented in this part of the project.

6. RTTs from the DNS server

For each DNS query the program measures the round-trip time (RTT) between sending the query packet and receiving the response. A high resolution timer is used to record the start time before sending the packet and compute the elapsed time after the response is received.

The RTT values are printed after each query and represent the network latency between the client and the DNS server being queried.

7. RTT from the requested domain

After the final DNS resolution step, the program performs an HTTP request to the resolved IP address. The RTT for this request is measured in the same way as the DNS queries. The time between sending the HTTP request and receiving the first response data is calculated and printed as the RTT for the requested domain.